Test 5: malla, contorno y evidencia

Nombre:Vadía Aguilar

1. Problema y objetivo

¿Basta el acuerdo en los nodos para aceptar el campo de desplazamientos de una barra axial calculado con MEF, y qué cambia al añadir $u(L) = 0$? Se compara el solver NumPy suministrado (2, 4 y 8 elementos lineales) con una referencia analítica independiente y se contrasta con el modelo alterado `BC_ALTERADO`.

2. Datos, supuestos y método

Barra elástica lineal con EA constante, $L = 2000\,\mathrm{mm}$, $E = 200\,000\,\mathrm{N/mm^2}$, $A = 1000\,\mathrm{mm^2}$, carga de extremo $P = 20\,000\,\mathrm{N}$ y carga uniforme $q = 10\,\mathrm{N/mm}$, ambas en $+x$ (N y mm). Contorno base: $u(0) = 0$ y $EA\,u'(L) = P$ (Fig. 1a). El problema es $-(EA\,u')' = q$ [2]; integrando con ese contorno,

$$u(x) = \frac{(P + qL)\,x - qx^2/2}{EA}, \qquad u(L) = \frac{PL + qL^2/2}{EA} = 0.3\,\mathrm{mm}. \tag{1}$$

El sistema $\mathbf{Ku} = \mathbf{f}$ se resuelve con `numpy.linalg.solve` [1]; las reacciones se recuperan como $\mathbf{r} = \mathbf{Ku} - \mathbf{f}$ con $\mathbf{K}$ y $\mathbf{f}$ originales (sin condensar): el residuo debe ser nulo en los grados libres y valer la reacción en los restringidos. El error de campo es el máximo de $|u_h - u_{\mathrm{ref}}|$ en los puntos medios, con u_h interpolado linealmente en cada elemento. Control numérico: $|R + P + qL|/|P + qL| \le 10^{-8}$ (comprobación de cálculo, no validación física). Se conservó `barra_mef.py`; el aporte propio es `analisis.py` (referencia y error), `figuras.py` y las pruebas `test_analisis.py`.

3. Resultados y verificación

La Tabla 1 y la Fig. 1b provienen de `salidas/resultados.csv` (`python src/ejecutar.py`). Las pruebas del solver y del análisis pasan; el residuo en grados libres es $\le 6 \times 10^{-11}\,\mathrm{N}$ en las tres mallas y el balance relativo es $\le 10^{-15}$.

Cuadro 1: Resultados. R_{izq} y R_{der} en N; error en puntos medios en mm; balance relativo $|R + P + qL|/|P + qL|$ adimensional. En `BC_ALTERADO` no hay error de campo comparable, pues la referencia es la del problema base.

Caso	n	$u(L)$ [mm]	R_{izq} [N]	R_{der} [N]	Balance rel.	Error medio [mm]
M2	2	0.3000	-40000	0	0.0e+00	6.250e-03
M4	4	0.3000	-40000	0	5.5e-16	1.563e-03
M8	8	0.3000	-40000	0	9.1e-16	3.906e-04
BC_ALTERADO	8	0.0000	-10000	-30000	0.0e+00	–

4. Interpretación y conclusiones

Acuerdo nodal frente a campo. Lo predicho antes de ejecutar (`BITACORA.md`) se cumple: $u(L) = 0.3\,\mathrm{mm}$ y todos los nodos coinciden con la referencia en las tres mallas, porque con EA

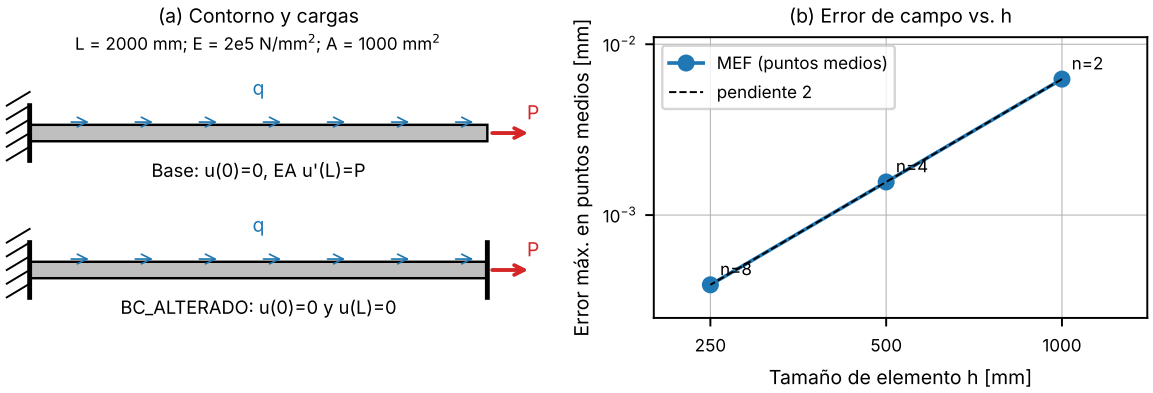


Figura 1: (a) Esquema del contorno base y del alterado, con cargas y unidades. (b) Error máximo en puntos medios frente al tamaño de elemento $h = L/n$; la línea discontinua tiene pendiente 2.

constante y cargas nodales consistentes el MEF lineal es exacto en los nodos. Aun así el error entre nodos no es nulo: u es cuadrática y u_h lineal por tramos, y el máximo en el punto medio es $qh^2/(8EA)$ (6,25e-3, 1,56e-3 y 3,91e-4 mm), que cae $\times 4$ al duplicar n (Fig. 1b). El acuerdo nodal, por tanto, no certifica la exactitud del campo.

Contorno. Al añadir $u(L) = 0$ el balance también cierra ($R_{\text{izq}} + R_{\text{der}} = -40000$ N), pero con $R_{\text{izq}} = -qL/2$ y $R_{\text{der}} = -(P + qL/2)$: la nueva reacción absorbe P y $u(L)$ pasa de 0,3 a 0 mm. Es un problema hiperestático distinto, no un refinamiento; cerrar el equilibrio solo prueba que $\mathbf{K}$ y $\mathbf{f}$ son coherentes, no que el modelo represente el problema pedido.

Conclusión. Los nodos coinciden con la solución exacta, pero el campo entre nodos converge solo con orden 2 en h. Una restricción adicional cambia el problema con equilibrio perfecto: el balance verifica el cálculo, no el modelo. Alcance: 1D lineal, EA y q constantes; sin validación experimental.

Declaración de uso de IA

Se usó Claude, el 5-nov-2026, como asistente para redactar `analisis.py`, `figuras.py`, `test_analisis.py`, este informe y los registros. Comprobación: pruebas automáticas, contraste con la fórmula cerrada de la Sec. 2 y revisión propia de cada bloque (detalle en `USO_IA.md`). Las predicciones de `BITACORA.md` se registraron antes de ejecutar el código.

Referencias

[1] NumPy developers. numpy.linalg.solve. `https://numpy.org/doc/stable/reference/generated/numpy.linalg.solve.html`. Consulta: 26 de octubre de 2026.

[2] TU Delft. Computational modelling: Rod. `https://interactivetextbooks.citg.tudelft.nl/computational-modelling/structural_linear/rod.html`. Consulta: 26 de octubre de 2026.